

BOB-093 “rutracker ReDoS regex bounds: measured timings

Revision: 5 **Last modified:** 2026-08-21T00:00:00Z **Scope:** the substantive (offline-provable) half of BOB-093. The live compose bring-up half is explicitly DEFERRED “ see “Deferred” below.

What the prior evidence bundle already established

docs/qa/BOB-093/evidence.txt + verdict_GREEN.json (2026-08-19, commit 1c0389a) establish, with runtime-class evidence:

1. re_search_queries was changed from the quadratic `<a.+?href="tracker\.php\?(.*?start=\d+)"` to the bounded `<a[^>]{0,512}?href="tracker\.php\?(["]{0,256}?start=\d+)"`.
2. That bounded form is present in the DEPLOYED plugin inside the running qbittorrent-proxy container (/config/qBittorrent/nova3/engines/ rutracker.py, sha256 76a6bd2eâ€¦) “ the Â\$11.4.108 layer-3 claim.
3. A self-validated challenge (challenges/scripts/rutracker_redos_regex_bounds_challenge.sh) with golden-good/golden-bad and a Â\$11.4.115 RED on a mutated copy.

What it did NOT establish, and what this round adds:

- Every check in that bundle is a **pattern-PRESENCE** check (does the deployed file contain the bounded string?). None of them **times** a regex, so none of them can see catastrophic backtracking in a regex that was never inspected. It covered ONE of the pluginâ€™s four regexes.
- The bundleâ€™s line “*proves deployed rutracker plugin functions end-to-end with the bounded regex*” is **not supported by its own payload**. In live_search_smoke.txt the single returned result carries "tracker": "linuxtracker", and rutrackerâ€™s own entry reads "status": "empty", "results_count": 0, "duration_ms": 164. The BOB-136 adoption audit reached the same conclusion independently.

Method

Growth-curve measurement, not inspection. Each regex is run against an adversarial input at doubling sizes; a pathological regex shows superlinear growth, a bounded one plateaus.

Host safety (Â\$12.6/Â\$12.12): every probe ran single-threaded under nice -n 19 ionice -c 3 timeout --signal=KILL <20â€“180>s. An OS-level

timeout is the only sound bound here – Python signal handlers cannot interrupt a running re.search, which is one uninterruptible C call, so an in-process alarm would not have fired. Probes print one flushed line per size, so a SIGKILL mid-run still leaves the completed rows as evidence. Killed in a table below is the hard bound firing, not a crash.

RED – before (all four regexes measured)

Growth ratio is time(2n)/time(n): ~2 = linear, ~4 = quadratic, ~8 = cubic.

**re_threads = <tr id="trs-tr-\d+.*?</tr> (re.S) –
QUADRATIC**

Attack '<tr id="trs-tr-1' * k: many row-starts, no </tr> anywhere, so every start scans to EOF. Applied to the WHOLE page via .findall().

k	bytes	seconds	ratio
500	8 000	0.113842	–
1 000	16 000	0.191936	1.69
2 000	32 000	1.002507	5.22
4 000	64 000	4.483936	4.47
8 000	128 000	13.434783	3.00

re_torrent_data – CUBIC (worst)

Attack 'a data-topic_id="1"' + "><" * k: every > and every < is a backtrack point for .*?> and .+?< under re.S. Run once **per row**.

k	bytes	seconds	ratio
250	519	0.171091	–
500	1 019	2.604068	15.2
1 000	2 019	25.804848	9.91
2 000	4 019	Killed at 40s bound	

A **2 KB** row cost **25.8 s**. Second attack, data-ts_text="1" storm ('a data-topic_id="1">t<' + 'data-ts_text="1"' * k), attacking the five inter-cell gaps: 0.98 s @ 4 KB, 8.62 s @ 8 KB, 29.17 s @ 12 KB, 65.13 s @ 16 KB. Third attack, leechmed storm: 10.27 s @ 72 KB.

re_search_queries (bounded by the earlier BOB-093 fix) – LINEAR –

Attack '<a ' * k. Historical form shown for contrast.

k	bytes	historical	.+?	ratio	current	bounded	ratio
2 000	6 000	0.063239	–	0.008630	–		
4 000	12 000	0.242998	3.84	0.018048			2.09

k	bytes	historical	.+? ratio	current	bounded ratio
8 000	24 000	1.254772	5.16	0.042664	2.36
16 000	48 000	3.930997	3.13	0.087473	2.05
32 000	96 000	<i>Killed at 20s</i>	â€œ	0.173063	1.98

The earlier BOB-093 fix is confirmed effective by measurement, not only by pattern presence. This is the negative result the item needed.

re_magnet = magnet:\?xt=urn:btih:([a-fA-F0-9]{40}) â€œ
LINEAR âœœ...

Attack: 39 valid hex chars + 1 invalid, repeated (near-miss at the {40} boundary). 0.001873 / 0.003536 / 0.007466 / 0.016386 / 0.034712 s at 120 KB â†’ 1.92 MB; ratios 1.89 / 2.11 / 2.19 / 2.12. Safe as written.

The fix

The decisive finding is that the dominant cost in `re_torrent_data` was **not** the inter-cell gaps but `.+?>` and `.+?<`: under `re.S` those two can span an entire `><` storm and permute against the five later gaps. Replacing them with negated classes that structurally cannot cross the delimiter they are searching for removes the permutation outright.

```
re_threads      <tr id="trs-tr-\d{1,12}\{0,4096\}?</tr>
re_torrent_data a data-topic_id="(P<id>\d{1,12})"["^<>]{0,512}?>(P<title
                  .{1,512}? data-ts_text="(P<size>\d{1,20})"                â€¦
```

Bound calibration (evidence, not guesswork). Largest values observed across every row fixture the repo ships (tests/unit/test_plugin_rutracker.py + tests/unit/merge_service/test_private_tracker_html_fixtures.py): row body **315**, topic-anchor gap **64**, title **30**, inter-cell gaps **18**, topic id **5 digits**. Against those the chosen bounds are 13—/8—/34—/28—.

Those multiples overstate the real headroom, and the honest figures are smaller. The shipped fixtures are synthetic and small. Independent review reconstructed a realistic `rutracker` row â€œ 1 415 B, 242-char Cyrillic title, full 10-column markup â€œ and against *that* the two tightest margins are **2.9— on the 4096 row bound and 2.6— on the 512 inter-cell gap bound**, not 13— and 8—. Both bounds are therefore âœœ~2âœœ3— a realistic rowâœœ, and neither has the comfortable margin the fixture-relative multiples suggest. This is why the bound-exceedance telemetry below exists: the margins are thin enough that a real page crossing them is a reachable state, so a crossing must be observable. Cost is linear in the bound, measured, so widening later is a known-cost change: `re_threads` @ `k=8000` â†’ 3.77 s (`b=8192`), 1.44 s (`b=4096`), 0.72 s (`b=2048`); inter-cell gap on a 256 KB chunk â†’ 3.22 s (2048), 0.21 s (1024), 0.017 s (512).

GREEN “ after, same inputs

attack	regex	before	after
gtlt_storm k=1000 (2 KB)	re_torrent_data	25.804848 s	0.000004 s
gtlt_storm k=16000 (32 KB)	re_torrent_data	<i>unreachable</i>	0.000048 s
ts_text_storm k=16000 (256 KB)	re_torrent_data	65.13 s @ 16 KB	0.027943 s
leechmed_storm k=16000 (144 KB)	re_torrent_data	10.27 s @ 72 KB	0.000133 s
tr_storm k=4000 (64 KB)	re_threads	9.4608 s	0.7887 s
tr_storm k=8000 (128 KB)	re_threads	<i>unreachable</i>	1.4393 s

re_torrent_data is now **flat** “ cost stops growing with input, because no quantifier can scan past its bound. re_threads is linear (min-of-3 across k=1000–8000: ratios 1.51 / 1.90 / 1.82 vs the pre-fix 2.96 / 4.14).

Correctness on real-shaped input: the bounded patterns extract byte-identical groups to the pre-fix patterns on every row fixture across both test files (**16** rows; this round first counted 14 “ the difference is whether deliberately-malformed rows are counted, cosmetic either way), and independent review reproduced that on a reconstructed realistic 1 415-byte row with a 242-char Cyrillic title: identical extraction.

They are NOT semantically equivalent in general, and that is stated as a fact rather than glossed. Review built 18 adversarial inputs and diffed old-vs-new match behaviour mechanically: **8 diverge**. Every divergence falls into one of two classes “ (a) the old pattern’s match was itself garbage (e.g. a title beginning with < yielded the old title 'Beatles', i.e. a raw markup shipped as a display name), or (b) a bound exceedance. **None converts a previously-CORRECT extraction into a wrong one.** Class (b) is the one with an operational cost, and it is what the telemetry below makes visible.

Suite state: 96 passed in tests/unit/test_plugin_rutracker.py. The merge_service parser figure reproduces as 31 + 25 when the two parser files are run **separately**; run together they hit a pre-existing collision (see Honest gaps).

Second affected file (a fork, \$11.4.251)

download-proxy/src/merge_service/search.py:1393 carried a **byte-identical copy** of both vulnerable regexes, and it “ not the qBittorrent plugin “ is the parser on the live :7187 search path the prior smoke exercised. Both copies are fixed, and

test_merge_service_fork_stays_bounded guards the fork so it cannot drift back alone.

Bound-exceedance telemetry (added after review, IMPORTANT-1)

Bounding a regex trades a hang for a **silent drop**, and both consumers were if match: with no else “ plugins/rutracker.py and download-proxy/src/merge_service/search.py, the latter logging only from *inside* a matched row. A legitimate row over 4096 B (long title + long forum name + long username) therefore vanished with **zero signal**: the “11.4.201 false-negative shape, and with only 2.6–2.9— real headroom it is a reachable state, not a theoretical one.

Both consumers now count what each stage should have produced and logger.warning on divergence:

- <tr id="trs-tr-\d row-starts vs re_threads matches “ row bound exceeded or unterminated row;
- re_threads matches vs re_torrent_data matches “ malformed row or field bound exceeded.

This converts a silent skip into an observable signal **and** makes the bounds calibratable from production logs instead of from synthetic fixtures “ which is the standing gap, since no real rutracker page is obtainable (below).

Both directions of the counter are pinned. UNDER-counting was already caught by the bound-exceedance tests. OVER-counting was **not**: review mutated re_row_start from <tr id="trs-tr-\d to a bare <tr and all 15 telemetry+ReDoS tests passed, because no healthy fixture contained a non-row <tr>. A real page has header rows and other tables, so that mutation would emit a spurious drop-warning on **every** page with nothing failing “ a cry-wolf regression vector. The healthy-page fixture now carries a <tr class="tbs-top"> header row, and the mutant is caught (assert 3 == 2).

Proven, not asserted: TestBoundExceedanceTelemetry (4 tests) drives __execute_search with a 5 000-byte row (verified to actually overrun the bound, else the test would prove nothing) and with a row the field parser rejects, and asserts the exact warning text; a fourth test asserts a clean 2-row page produces **no** drop warning (“11.4.201(1) “ telemetry that cries wolf is a FAIL-bluff), and a fifth guards the fork’s copy.

Guards

Structural “ tests/unit/

test_plugin_rutracker.py::TestReDoSRegexBounds (11 tests, 2.5 s).

Flags any */+ quantifier whose atom is . or a negated class [^“|] “ the atoms that can scan to EOF. \d+ / [-\d]+ are digit-runs, self-limiting,

allowed. Self-validating on every run against the four real pre-fix patterns as

golden-bad, plus a false-positive guard on benign bounded/escaped forms (§11.4.201(1)).

Behavioural “tests/stress/test_rutracker_redos_bounds.py (3 tests, ~24 s). Times the live compiled regex against each attack and caps it; the same harness must simultaneously measure the pre-fix pattern **over** the cap, so a broken timer cannot pass everything (§11.4.107(10)). A source check alone is source-class evidence for a runtime-class claim (§11.4.226) and would miss a differently-spelled unbounded scan.

Every teeth-check is relative or floor-shaped, none is an absolute floor on hist_s. The two flat cases keep their absolute $\text{live_s} \leq \text{cap}$ regression assert, but their *teeth-check* “the half that proves the harness can still see the blow-up” is $\text{hist_s} > 100 \times \text{live_s}$.

The first version used an absolute $\text{hist_s} > \text{cap}$ floor, and that was the same defect as the `tr_storm` cap, mirrored. Re-review measured the historical `gtlt_storm` at **0.4887 s** in a full-suite run and 0.465 / 0.453 / 0.452 s standalone “all **under** the 0.5 s floor, so the teeth-check **false-FAILED on healthy code**. Cause established, not guessed: the 1.70 s “idle” baseline was taken with cores at powersave clocks under contention; with load down and a core boosting to 2.7 GHz the same measurement runs **~3.7× faster**. A 3.4× floor margin cannot survive 3.7× clock-state variance, and `ts_text_storm` was the next domino at 5.4×.

Reproduced here across three consecutive runs on this box: historical `gtlt_storm` came in at **0.6762 / 0.5190 / 0.8430 s** “run 2 sat 3.8% above the old floor, i.e. a coin flip. Over the same three runs the *separations* were 846 316× / 467 986× / 602 566×, never near the 100× threshold. Both halves share the clock, so the ratio cancels the variance “the identical argument already used for `tr_storm`, now applied consistently.

Then a **fourth** box state appeared during round-3 review that neither of us had seen “load ~1.5 with cores at **3.4 GHz**, faster than the 2.7 GHz state that produced this finding. Historical `gtlt_storm` measured 0.571 s and 0.563 s there: **the retired 0.5 s floor would have been a coin flip again**, in a state it was never tuned for. The relative form read 831 717×. Across all four states the separations are 467 986 “846 316× (`gtlt_storm`) and **566 “1 052×** (`ts_text_storm`), so the 100× threshold sits **5.7×** above the worst arm’s worst-ever reading.

Note what review did rather than hide: it left the honest FAIL artifact in `qa-results/` instead of re-running until green (§11.4.248).

`tr_storm` uses a **relative discriminator** instead, and the original absolute cap on it was a defect. There the fix is *linear* and the pre-fix is *quadratic*, so the honest gap is ~15×, not ~80× “and both halves inflate together under load. Independent review measured $\text{live_s} = 4.13 \text{ s}$ against a 0.98 s idle baseline (**4.2× inflation under 3-agent load**) and the 3.5 s cap **failed with zero code defect**: a §11.4.201(1) FAIL-bluff, and precisely the

re-run-until-green trainer (§11.4.248) that the removed ratio test had been. The comment budgeted ~2s slowdown; this box does over 4s.

The fix is to compare the two halves measured in the **same run**, which cancels the shared inflation: `hist_s > 4 - live_s`, plus a loose 10 s sanity ceiling that is not the discriminator. Separations: idle 14.6s; under 3-agent load 9.2s; a quadratic regression 1.09s. The 4s threshold sits with >2s margin on both sides.

The relative arm also carries its own teeth-check, `hist_s > 1.0 s`. Without it a zeroed timer read as `float("inf")` separation and **passed**: review stubbed `_best` to 0.0 (mutation M-D1) and, while the two absolute arms failed, the relative arm sailed through, because it had dropped the teeth-check the absolute arms kept. Suite-level the dead-timer control still held via the siblings, so a broken timer cannot pass everything stayed literally true but that *arm's* own §11.4.107(10) self-validation was gone. Observed `hist_s` range across every box state is **3.08s–38.1 s**, so a 1.0 s floor keeps ~3.1s margin. Provenance, because this figure moved twice: round-3 review recorded four max-boost samples (3.29 / 3.82 / 4.11 / 4.18 s) and asked for the range to be refreshed from an older 3.64 s low; while applying that refresh, the very next verification run here measured 3.1290 s, so an 8-sample probe was taken in the same state and extended the low again to **3.084 s** (3.084s–3.891 s, min-of-8, load ~1.44). Neither correction changes a decision but the floor stays 1.0 and the flat threshold stays 100 but writing 3.29 s while holding a 3.08 s measurement would be recording a number already known to be superseded.

This is the one absolute constant left in the harness, so its exposure is stated rather than assumed. It is **not R2-1 repeating**, for a structural reason: the low was measured **at max boost**, so unlike the retired 0.5 s floor there is no faster state on this hardware waiting to cross it, and residual intra-state noise is 1.26s against a 3.1s margin. Its *value* is also not load-bearing for the role it exists for but review weakened it 1000s to 0.001 s and the dead-timer and broken-payload classes were still caught, since they sit 5–6 orders below any sane floor; it sits mid-way in a ~3-order-wide valid band. Known cost, accepted: a ~3.1s-faster single-thread host would false-FAIL it **loudly**, never silently, and the derivation above makes the retune straightforward. Applying the flat arms' relative teeth closes their dead-timer path by the same mechanism, since `0 > 100 - 0` is False.

A growth-**ratio** test (`time(2n)/time(n)`) was written earlier and **removed, not weakened**, for the same class of reason but a ratio of two *independent* measurements amplifies noise. The relative discriminator here is different in kind: it ratios two measurements of the *same* input in the *same run*, so contention is common-mode and cancels rather than compounding.

§1.1 mutation but the guards have teeth

The mutation is the fix's own revert (§11.4.115(F)), applied to both files.

mutation	structural (unit)	behavioural (stress)	telemetry (unit)
revert both regexes to pre-fix	4 failed	4 failed	â€œ
C: {0,4096}? â†’ {0,999999}?	11 passed â€œ blind by design	1 failed: separation 1.35Ã— — vs 4.0Ã— threshold (live 3.2316 s vs historical 4.3681 s)	â€œ
strip both logger.warning telemetry blocks	â€œ	â€œ	3 failed of 4
M-D1: _best â†’ 0.0 (dead timer)	â€œ	3 failed of 3 (was 2 of 3)	â€œ
M-D2: payload builders â†’ ""	â€œ	3 failed of 3, deterministically (was stochastic)	â€œ
M-T2: re_row_start â†’ bare <tr>	â€œ	â€œ	1 failed (was 0 of 15)

Mutation C is the load-bearing demonstration that the two layers are **complementary, not redundant**: a bounded *spelling* with an unbounded *effect* is invisible to a pattern-text rule by construction, and only the behavioural timer catches it. Review found the same independently. In the telemetry row the 4th test correctly still passes â€œ it is the false-positive guard, which *should* pass when no warning is emitted at all.

Reviewer-run mutations (five, independent of mine): none escaped both nets.

Restored afterwards and verified by sha256 against the pre-mutation baseline (both files OK), with a residue scan showing 0 pre-fix patterns left in source (Â§11.4.84).

Stability (Â§11.4.50): 3 consecutive full runs of the behavioural guard â€œ 3 passed / 3 passed / 3 passed.

Deferred â€œ and why

The itemâ€™s title asks for a **live compose bring-up**. That is **not done here**, deliberately: container orchestration is owned by the operatorâ€™s `start.sh` (Hard Stop #3), the stack is currently serving, and this agent must not bring it up or restart it. Concretely deferred:

1. `./install-plugin.sh + ./start.sh --reload-plugins` to copy the fixed plugins/rutrackers.py into config/qBittorrent/nova3/engines/rutrackers.py and load it. The installed copy on disk is

- therefore still the pre-fix one; it was left untouched on purpose, since a direct edit there is clobbered by the next install and the source of truth is `plugins/`.
2. `./start.sh --reload-python` for the `merge_service/search.py` change (bind-mounted, needs a `__pycache__` clear + container restart).
 3. **Re-running challenges/scripts/rutracker_redos_regex_bounds_challenge.sh** against the container to re-establish the `Â§11.4.108` layer-3 sha256 match for the two newly-fixed regexes. The existing GREEN verdict covers only `re_search_queries`.
 4. **The item's 4th acceptance sub-step "capture timing of a large rutracker result page (<2 s)"** still unmet, and it is not merely a deployment matter: rutracker returned `status: empty, results_count: 0` in the prior smoke, so no large page has ever been obtained to time. Independent review attempted a live fetch with the operator's cookies `rutracker.txt` and **rutracker.org now serves a Cloudflare JS challenge (Just a moment) to curl even WITH valid cookies**. That independently explains the prior smoke's `results_count: 0`, and it means this sub-step is **not merely blocked on credentials** cookies alone cannot satisfy it. Filed as **BOB-172**, and independently re-confirmed with cookie-bearing curl; deliberately not worked around here.

Everything above is layer-3/layer-4 (`Â§11.4.108`). This round's evidence reaches **SOURCE and behaviour of the compiled regex in-process** it does not claim the container is running the fix.

Honest gaps

- **Raising the bounds is not free safety, and that is why they were left alone.** A larger bound linearly raises the *adversarial* budget the bound exists to cap measured: `b=8192` costs 3.77 s worst case, `~2.6`— the current `b=4096`. Raising on a hypothesised row shape with **zero observed instances** would trade a MEASURED DoS budget for an UNMEASURED truncation risk: a `Â§11.4.6` guess in the opposite direction. With the telemetry now emitting exact counts, keep-and-calibrate-from-observation is the `Â§11.4.101` reversible-safe default. The calibration debt is tracked as its own item on BOB-093 rather than living only in this sentence.
- **No real rutracker page was available to calibrate against**, and the Cloudflare challenge (BOB-172) means one cannot currently be fetched. Bounds are calibrated from synthetic fixtures plus a reconstructed realistic row, giving only `2.6`—`2.9`— real headroom on the two tightest bounds. A row crossing a bound is **no longer silent** (telemetry above), but it is still **dropped**, not recovered the warning tells you to raise the bound, it does not save that row. Raising the bounds is a one-literal change with its cost curve recorded beside it.
- **re_threads is bounded, not free.** A 128 KB adversarial page still costs `~1.4` s. The win is linear-instead-of-quadratic; the constant scales with the bound.

- The `{1,512}`? inter-cell gaps remain `.`-based. They are bounded and measured flat, but a negated-class formulation is impossible there because real markup between cells legitimately contains `<`, `>` and `"`.
- **Unrelated pre-existing breakage observed, not fixed:**
 1. the schemathesis pytest plugin fails to import (`ModuleNotFoundError: No module named 'rpd.rpd'`), so every pytest run above used `-p no:schemathesis`;
 2. the BOB-165 recipe `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1` breaks 5 pre-existing env-dependent tests in `tests/unit/test_plugin_rutracker.py` (`TestConfig::test_default_mirrors`, `test_env_mirrors_override`, `test_get_env_with_default`, `test_get_mirrors_from_env_empty`, `test_get_mirrors_from_env_whitespace`) – they need an autoloaded env plugin. Proven pre-existing: HEAD’s own copy of the file fails the same 5 under the same flags. With autoloading on and only schemathesis disabled the file is 96/96 green. It also requires `-p pytest_timeout`, since the `--timeout=60` in `pyproject.toml` is otherwise an unknown argument;
 3. `tests/unit/merge_service/test_html_parsers.py` + `test_search_coverage.py` run together produce 31 setup errors (`No module named 'merge_service.search.deduplicator'`; `'merge_service.search' is not a package`). Proven pre-existing: with `search.py` reverted to its HEAD content the identical 31 errors appear.
- `plugins/rutracker.py` does not satisfy `ruff` format; that drift is pre-existing on HEAD and was left alone to keep the diff scoped.